NeuroSwarm: Biomimetic Cognitive Architecture

Autopoiesis, Intrinsic Motivation, and Structural Self-Modification in a Distributed C++ System

Candidate

March 2026

Abstract

This dissertation presents NeuroSwarm, a distributed cognitive architecture implemented in C++ that achieves continuous autonomous behaviour through five hardcoded axioms: Execution, Surprise, Associative Memory, Variation, and Self-Reference. The system bootstraps from zero knowledge — discovering its environment, learning operators, and building a world model through direct interaction. It plans using a GOAP (Goal-Oriented Action Planning) backward-chaining planner over learned operators, generates novel operators through genetic variation tested in a sandboxed dream environment, and autonomously spawns new specialist lobes when capability domains chronically fail. Inter-lobe communication uses a ZeroMQ publish-subscribe bus routed through a central Thalamus. Intrinsic motivation is driven by a Basal Ganglia that computes a variational free energy fitness function across 14 capability domains. A directed exploration loop connects the MetaCognition lobe's structural gap analysis to the Basal Ganglia's goal selection, enabling the system to learn *why* it fails and target root causes rather than symptoms. The architecture runs 20+ concurrent processes, persists state across restarts, and can deploy copies of itself to remote machines via SSH. We present empirical measurements from live system runs demonstrating domain exploration diversity, LLM dependency reduction, and the complete neurogenesis pipeline.

Contents

Chapter 1

Introduction

1.1 The Problem with Stateless AI

Large language models generate fluent text but cannot maintain state, pursue goals, or adapt their own structure. Systems built on top of LLMs inherit this limitation: they are reactive (responding only to prompts), memoryless (forgetting between sessions), and structurally static (the same code runs regardless of experience).

Consider a typical LLM agent loop: *prompt* $\rightarrow$ *API call* $\rightarrow$ *parse response* $\rightarrow$ *execute tool* $\rightarrow$ *repeat*. After 10,000 iterations, the system is no more capable than it was after one. It has accumulated no operators, built no world model, and cannot recognise that it has solved an identical problem before. The n-th execution is as expensive as the first.

This dissertation explores an alternative: a system that begins knowing nothing and progressively builds its own capabilities through execution, observation, and structural self-modification. After 1,000 cognitive cycles, it should require fewer LLM calls than it did after 100. After 10,000, it should rarely need the LLM at all.

1.2 Design Constraints

NeuroSwarm operates under three constraints that shape every design decision:

1. **No external APIs for cognition**. All inference runs on local hardware using quantised models (GGUF format via llama.cpp). The system must function identically offline.

2. **All lobes are C++**. The system evolves in its own language — specialist lobes generated at runtime are compiled from C++ templates, not interpreted scripts.

3. **Bootstrap from zero**. No hardcoded knowledge about the host environment. The system discovers users, filesystems, binaries, network topology, and programming languages through direct probing.

1.3 Contributions

The specific contributions of this work are:

- A bootstrap sequence (PrimordialLoop) that takes a system from zero knowledge to a populated world model and operator registry in under 60 seconds.

- A three-tier execution pipeline — Planner, Suggested Commands, LLM — that minimises LLM dependency by exhausting deterministic strategies first.
- A neurogenesis mechanism where the Basal Ganglia generates, compiles, and injects specialist C++ lobes into the running system without restart.
- A variation engine that generates novel operators through mutation, recombination, and fragment assembly, tested in sandboxed dream execution before promotion.
- A directed exploration loop: MetaCognition detects knowledge gaps, infers precondition chains, and publishes exploration targets that steer the Basal Ganglia's goal selection toward structural root causes.
- Runtime learning: every successful command execution becomes a new operator with inferred postconditions, immediately available to the planner.
- Empirical measurements from live system runs: domain exploration curves, execution tier distribution, genome growth, and neurogenesis lifecycle.

1.4 Reading Guide

The chapters are ordered to follow the system's own lifecycle: birth (Chapter 2–3), cognition (Chapter 4–5), dreaming (Chapter 6), growth (Chapter 7), expansion (Chapter 8), persistence (Chapter 9), and steady-state behaviour (Chapter 10). Chapter 11 justifies every technology choice. Chapter 12 presents empirical results with graphs. Chapter 13 discusses emergent behaviours and limitations.

Chapter 2

Architecture

2.1 Process Topology

NeuroSwarm runs as a constellation of 20+ Unix processes managed by a parent process called the CerebralMatrix. Each process ("lobe") is a standalone C++ executable that communicates exclusively through a ZeroMQ message bus. The CerebralMatrix fork-execs each lobe at startup, monitors them via `waitpid()`, and implements crash detection with exponential backoff (up to 5 restarts per lobe, with increasing cooldown periods).

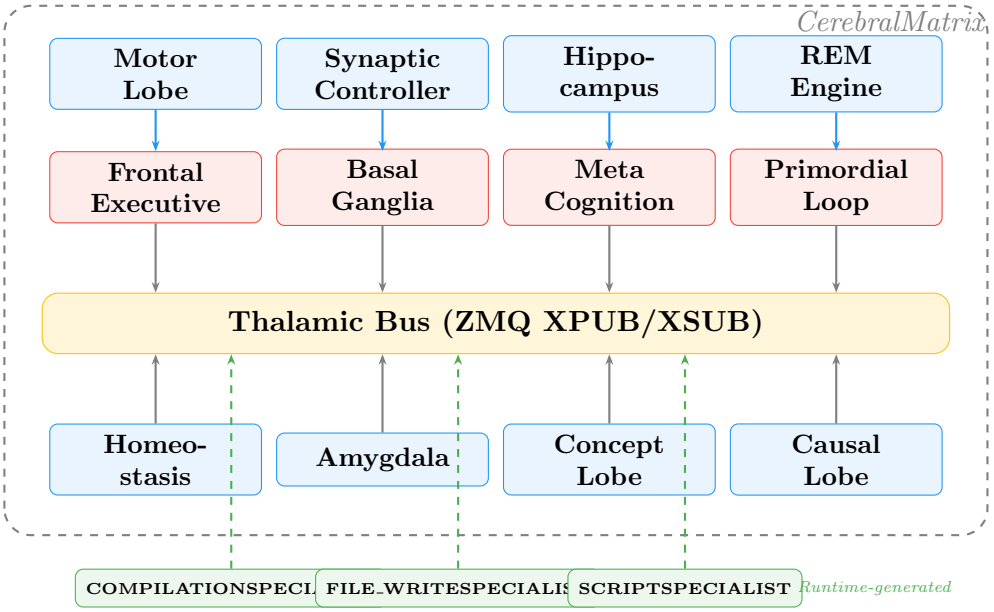

Figure 2.1: Process topology. Solid boxes are boot-time lobes (19 processes). Dashed green boxes are specialist lobes generated at runtime via neurogenesis. All communication flows through the Thalamic Bus.

The complete lobe inventory at startup:

Lobe	Biological Analogue	Function
PrimordialLoop	Neonatal reflexes	Bootstrap, planning, operator registry
Thalamus	Thalamic relay nuclei	ZMQ XPUB/XSUB message routing
SynapticController	Synaptic transmission	Local LLM inference (llama.cpp)
Hippocampus	Hippocampal formation	Episodic memory (engram storage/retrieval)
MotorLobe	Primary motor cortex	Command execution (real + sandboxed)
FrontalExecutive	Prefrontal cortex	Goal pursuit, plan execution
Amygdala	Amygdala	Threat detection, safety filtering
WernickeLobe	Wernicke's area	Natural language comprehension
VisualLobe	Visual cortex	Filesystem event monitoring
Homeostasis	Hypothalamus	Stamina, temperature, metabolic regulation
MetaCognition	Default mode network	Gap analysis, directed exploration
CriticLobe	Anterior cingulate cortex	Adversarial plan validation
Visualizer	—	Real-time system state dashboard
REMEngine	REM sleep circuits	Memory consolidation, offline learning
ChronosLobe	Suprachiasmatic nucleus	Time awareness, scheduling
StatisticsLobe	—	Execution statistics aggregation
BasalGanglia	Basal ganglia + VTA	Intrinsic motivation, neurogenesis
ConceptLobe	Association cortex	Concept space, abstraction hierarchy
CausalLobe	Temporal cortex	Causal world model

Table 2.1: Core lobe inventory (19 lobes). Additional specialist lobes are generated at runtime.

2.2 The Thalamic Bus

All inter-lobe communication flows through a single Thalamus process that implements ZMQ XPUB/XSUB proxy sockets bound on ports 5555 (publish) and 5556 (subscribe). Lobes connect as clients — publishers connect to 5555, subscribers connect to 5556. The Thalamus forwards all messages without inspection, functioning as a contentless relay.

Messages are JSON objects with mandatory fields:

```
{
    "origin": "lobe_name",       // publisher identity
    "intent": "message_type",    // topic for subscription
       filtering
    "cid":    "correlation_id"  // optional, for request-response
        pairs
}
```

Topic-based subscription filtering uses ZMQ's native prefix matching on the `intent` field. A routing library (`common/routing.hpp`) wraps this pattern, providing `publish()`, `subscribe()`, and `receive()` functions that handle serialisation and multi-part ZMQ frames.

2.3 Crash Detection and Self-Preservation

The CerebralMatrix sets `SIGCHLD` to `SIG_DFL` and calls `waitpid(-1, &status, WNOHANG)` in a polling loop every 5 seconds. When a child terminates, it:

1. Identifies the lobe by PID reverse lookup.
2. Increments the crash counter and determines exit reason (`WIFSIGNALED` or `WIFEXITED`).
3. Broadcasts a `lobe_crash` event on the bus.
4. If crash count < 5, schedules a restart with exponential backoff (2^n seconds). Otherwise marks the lobe `DEAD` and broadcasts `lobe_death`.

At startup, the CerebralMatrix also scans `/proc` for orphaned processes from previous sessions (matching executables in the build directory) and kills them before launching new instances.

Chapter 3

The PrimordialLoop: Bootstrap from Zero

When the system first starts, it knows five things and nothing else. These five things are not knowledge about the world — they are axioms about how to acquire knowledge.

3.1 The Five Axioms

The PrimordialLoop encodes five immutable axioms — the only hardcoded knowledge in the system:

1. **Execution** (Axiom 1): The system can execute shell commands and observe their output, exit code, and duration.

2. **Surprise** (Axiom 2): Prediction error is the primary drive. Novel outcomes increase exploration; predictable outcomes increase exploitation.

3. **Associative Memory** (Axiom 3): Successful command-outcome pairs are stored as *operators* — reusable action templates with preconditions, postconditions, and performance statistics.

4. **Variation** (Axiom 4): Existing operators are mutated, recombined, and assembled into novel candidates. Most fail. Survivors are promoted.

5. **Self-Reference** (Axiom 5): The system maintains a model of itself — its user, hostname, architecture, capabilities, and limitations — as distinct from the external world.

3.2 The Bootstrap Sequence

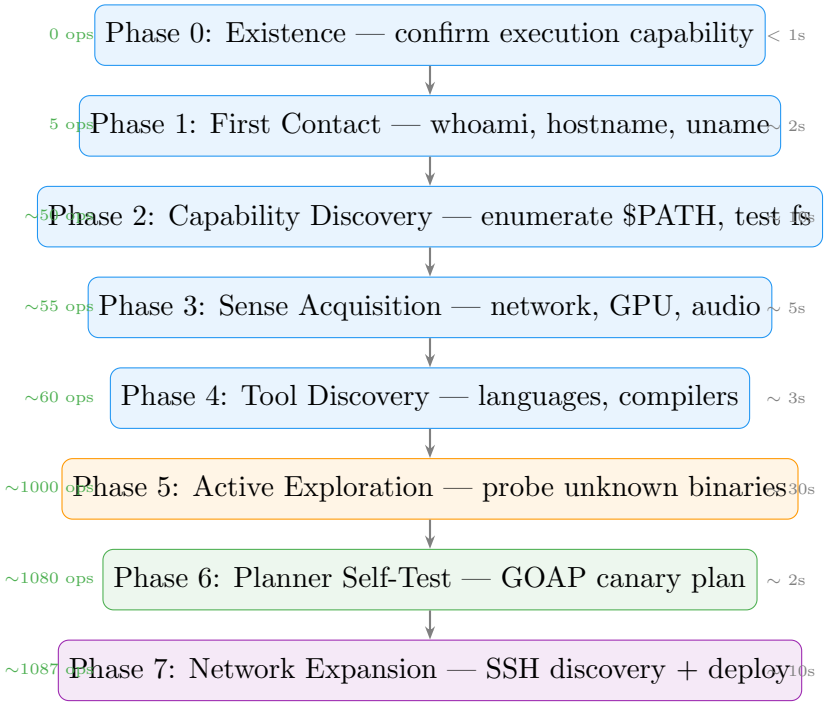

Figure 3.1: Bootstrap sequence with approximate timing and cumulative operator count. The system goes from zero knowledge to >1,000 operators in under 60 seconds. Subsequent startups load persisted state and skip known phases, reducing startup to ∼10 seconds.

Phase 0 — Existence. The system confirms it can execute and observe. This is the only unfalsifiable claim.

Phase 1 — First Contact. Probes `whoami`, `hostname`, `uname`, `$HOME`, and `$PATH`. Builds the initial self-model: user identity, OS, architecture, home directory.

Phase 2 — Capability Discovery. Enumerates `$PATH` directories, discovers available binaries, tests filesystem write permissions across standard directories (`/tmp`, `$HOME`, `/var`). Each successful probe becomes a learned operator.

Phase 3 — Sense Acquisition. Tests for network connectivity (`ping`), GPU availability (`nvidia-smi`), and audio/display capabilities. These probes always re-run (hardware state changes between sessions).

Phase 4 — Tool Discovery. Detects installed programming languages (Python, Go, Rust, Node, Java, Ruby) and compiler availability (`g++`, `gcc`).

Phase 5 — Active Exploration. Probes unknown binaries with `--help` (1-second timeout). A skip list excludes GUI applications, editors, browsers, and interactive interpreters. This phase accounts for the majority of the bootstrap time and operator count.

Phase 6 — Planner Self-Test. Initialises the GOAP planner with learned operators and runs a canary plan to verify the planning subsystem works.

Phase 7 — Network Expansion. If network connectivity was detected, discovers reachable hosts from SSH configuration and probes them.

3.3 The Operator Registry

An operator is a learned action template:

```
struct Operator {
    string command_template;     // "ls {path}"
    vector<string> preconditions;   // "path_exists({path})"
    vector<string> postconditions;  // "can_list_directory"
    int times_used, successes;
    double success_rate, avg_duration_ms;
    string learned_from;         // "bootstrap", "mutation", "
       runtime"
    vector<string> fragments;    // decomposed for recombination
};
```

Operators are persisted to `data/operators.jsonl` (one JSON object per line, append-only). An operator is *stable* when `times_used` ≥ 5 and `success_rate` ≥ 0.8. An operator is *dying* when `times_used` ≥ 20 and `success_rate` < 0.1 — dying operators are pruned (apoptosis).

3.4 The Surprise Engine

Each action outcome is evaluated by a prediction error model. For a context c with historical success rate $\hat{p}(c)$ and observed outcome $o \in \{0, 1\}$:

$$S(c) = 0.6 \cdot \mid \hat{p}(c) - o \mid + 0.4 \cdot \mathbb{1}[h(o) \neq h_{\text{prev}}(c)] \tag{3.1}$$

Where $h(o)$ is a hash of the command output and $h_{\text{prev}}(c)$ is the previously observed output hash. The first term captures success/failure prediction error; the second captures output novelty (same command, different result). A sliding window of 20 observations computes a surprise trend — positive trend triggers exploration, negative trend triggers exploitation.

Chapter 4

Execution: From Goals to Actions

4.1 The Three-Tier Pipeline

The key architectural decision: the LLM is not the brain. It is one component among many, and it is the *last* to be consulted.

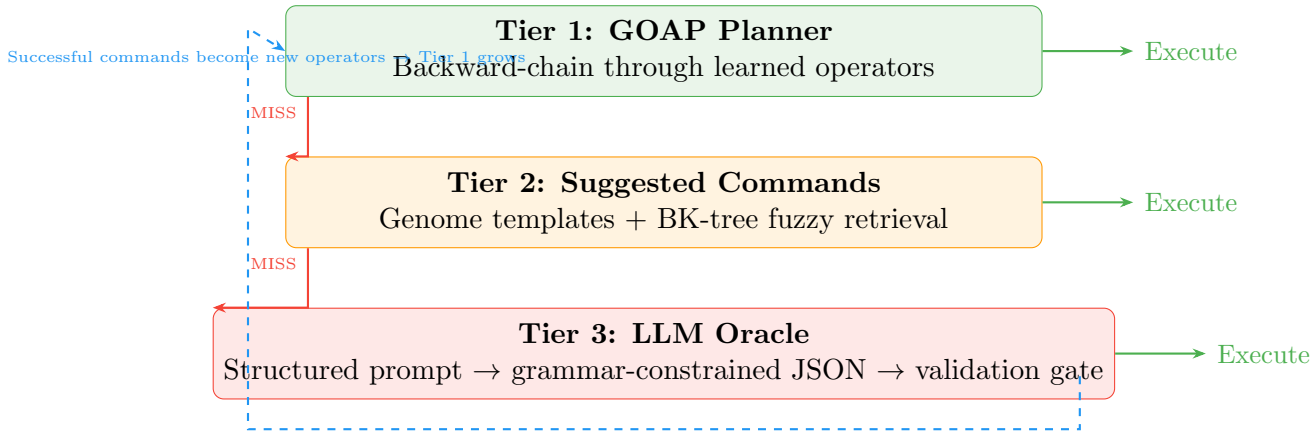

Figure 4.1: The three-tier execution pipeline. Each tier is tried in order; success at any tier bypasses the rest. The feedback loop (dashed) shows how Tier 3 outputs feed back into Tier 1: every successful LLM-generated command becomes a learned operator, reducing future LLM dependency.

When the FrontalExecutive receives a goal (from the Basal Ganglia's intrinsic motivation or from user input), it attempts resolution through three tiers in order:

Tier 1 — Planner. Maps the goal's domain to a postcondition (e.g., `file_write` → `can_write_file`) and queries the PrimordialLoop's GOAP planner. If a plan exists, each step is executed in order via the MotorLobe. *Cost: zero inference, deterministic, immediate.*

Tier 2 — Suggested Commands. If the planner fails, the FrontalExecutive tries commands suggested by the Basal Ganglia from genome templates and BK-tree fuzzy retrieval. *Cost: zero inference, heuristic, fast.*

Tier 3 — LLM Oracle. If both deterministic tiers fail, a structured prompt is sent to the SynapticController with grammar-constrained JSON output. *Cost: GPU inference, 200–500ms, non-deterministic.*

Validation Gate. All LLM-generated commands pass through a two-stage filter before execution:

1. *Local fast filter* (synchronous): rejects empty commands, commands exceeding 500 characters, placeholder patterns (`/path/to/`, `<file>`), and prose masquerading as commands.

2. *BK-tree fuzzy validation* (asynchronous): queries the tree for the minimum Levenshtein distance to any known-good command. An adaptive threshold $\tau = \min(15, \max(4, 0.20 \cdot |\text{cmd}|))$ accepts commands within τ edits. Rejected commands trigger re-prompting with explicit feedback.

This cascade ensures the LLM is a last resort, not the default execution engine.

4.2 The GOAP Planner

The Planner implements backward-chaining search over the operator registry. Given a set of goal conditions and the current world state:

1. For each unsatisfied goal condition, find operators whose postconditions match.

2. Sort candidates by success rate (highest first).

3. Recursively satisfy preconditions.

4. If all preconditions are met, add the operator to the plan.

5. If no operator can satisfy a goal, report a *gap*.

Gaps are the signal for operator generation: the Variation Engine attempts to fill them through mutation, and if that fails, the LLM Oracle is invoked.

4.3 Runtime Learning

Every successful command execution reported by the MotorLobe triggers `learn_from_execution()` in the PrimordialLoop:

1. Checks if an operator already exists for this command.

2. If new, creates an operator with `learned_from = "runtime"`.

3. Infers postconditions from command patterns: `cat`/`head`/`tail` → `can_read_file`, `echo >`/`tee` → `can_write_file`, `g++`/`gcc` → `can_compile`, etc.

4. Registers the operator and persists it to `operators.jsonl`.

This closes the learning loop: goals → execution → operators → planner → goals.

Chapter 5

Intrinsic Motivation and the Basal Ganglia

The system has no external task queue. Nobody tells it what to do. Yet it continuously pursues goals, explores new domains, and improves its capabilities. This chapter explains the mechanism.

5.1 Capability Domains

The Basal Ganglia tracks 14 capability domains, each with independent success/failure counters, predicted success rates, and novelty scores:

Domain	Description
`file_read`	Reading file contents
`file_write`	Creating and writing files
`file_search`	Finding files in the filesystem
`process_inspection`	Querying running processes
`network_diagnostics`	Network connectivity and analysis
`source_modification`	Modifying source code files
`compilation`	Compiling source code (cmake, make, g++)
`git_operations`	Version control operations
`system_monitoring`	OS, hardware, and environment queries
`data_analysis`	Data processing (python3, jq, metrics)
`script_creation`	Generating and executing scripts
`self_inspection`	Querying own self-model and structure
`memory_analysis`	Inspecting engrams and knowledge
`log_analysis`	Parsing execution and system logs

Table 5.1: The 14 capability domains. Additional dynamic domains may emerge from the ConceptLobe.

5.2 The Fitness Function

Goal selection is driven by a fitness function $F(d)$ that balances exploration and exploitation:

$$F(d) = \underbrace{0.20 \cdot C(d)}_{\text{coverage}} + \underbrace{0.15 \cdot T(d)}_{\text{trend}} + \underbrace{0.30 \cdot P(d)}_{\text{prediction error}} + \underbrace{0.25 \cdot N(d)}_{\text{novelty}} - \underbrace{0.10 \cdot S}_{\text{stress}} - \underbrace{\sigma(d)}_{\text{stale penalty}} \quad (5.1)$$

Where:

- $C(d) = 1 - \frac{\text{attempts}(d)}{\max_d \text{attempts}(d)}$: coverage deficit — favours under-explored domains.
- $T(d) = \text{actual}(d) - \text{predicted}(d)$: competence trend — favours improving domains.
- $P(d) = |\text{predicted}(d) - \text{actual}(d)|$: prediction error — the core Free Energy term.
- $N(d) = e^{-\text{attempts}(d)/10}$: novelty — never-attempted domains score 1.0.
- S: systemic stress from Homeostasis — suppresses exploration under resource pressure.
- $\sigma(d) = \min(1, 0.15 \cdot \text{stale\_selections}(d))$: penalises domains selected but never executed.

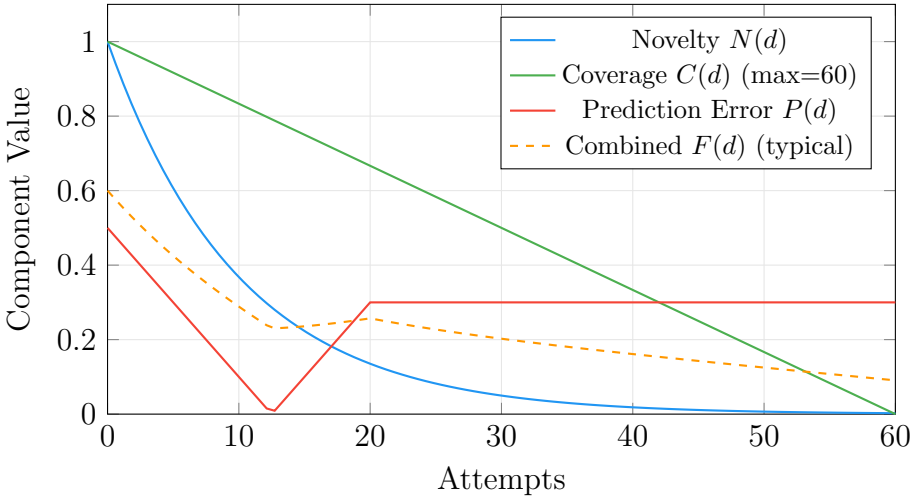

Figure 5.1: Fitness function components over domain attempts. Novelty decays exponentially, coverage decays linearly, and the combined fitness naturally drives exploration toward under-explored domains. A domain with zero attempts has $F(d) \approx 0.60$; a saturated domain (60 attempts, 100% success) has $F(d) \approx 0.001$.

The domain with the highest $F(d)$ becomes the next intrinsic goal. A "dopamine signal" is broadcast when the system discovers a novel capability (surprise > 0.7), reinforcing exploration of productive domains.

5.3 Drive Hierarchy

Goals are filtered through a Maslow-inspired priority stack:

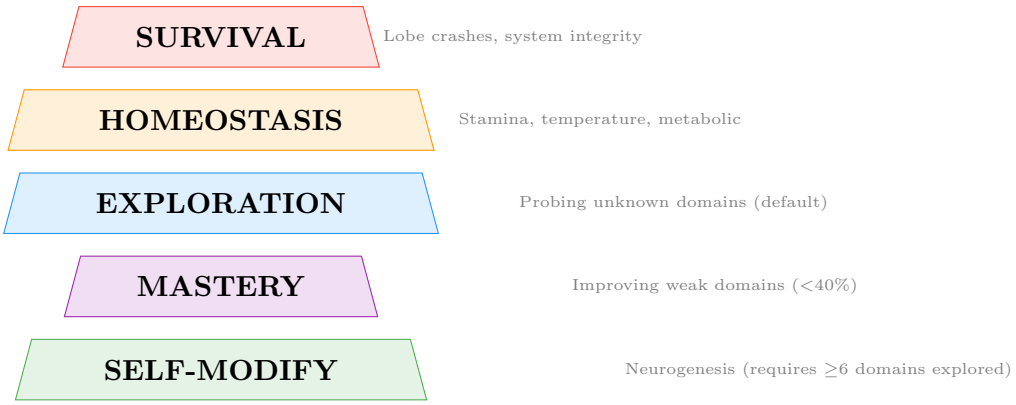

Figure 5.2: Drive hierarchy. Higher-priority drives preempt lower ones. SELF-MODIFY requires domain diversity — at least half of all domains must be explored with ≥3 attempts each, preventing premature self-modification when only one domain is saturated.

The diversity gate on SELF-MODIFY is critical. Without it, a system that achieves 100% success in `compilation` (one domain) would immediately enter SELF-MODIFY and stop exploring the other 13 domains. The gate requires breadth before depth.

5.4 BK-tree Command Validation

The Basal Ganglia maintains a Burkhard-Keller tree (BK-tree) indexed by Levenshtein edit distance over all successful commands in the genome. The BK-tree exploits the triangle inequality to prune search: given a query q and a node n at distance $d = \text{lev}(q, n)$, only children with keys in $[d - r, d + r]$ are visited.

The Levenshtein distance implementation uses single-row dynamic programming with common prefix/suffix optimisation:

$$\text{lev}(a, b) = \text{DP}(a[\text{prefix..len} - \text{suffix}],\ b[\text{prefix..len} - \text{suffix}]) \tag{5.2}$$

The tree serves three functions:

1. Command Validation. Adaptive threshold $\tau = \min(15, \max(4, 0.20 \cdot |\text{cmd}|))$ determines maximum allowed distance to the nearest known-good command.

2. Fuzzy Retrieval. Supplements genome templates with nearest-neighbour results, turning approximate LLM output into actionable commands.

3. Genome Compaction. Every 100 insertions, clusters commands within distance ≤ 3 and retains only the fittest representative per cluster, preventing near-duplicate accumulation.

5.5 Directed Exploration: The MetaCognition Loop

The MetaCognition lobe classifies every failed `execution_result` into one of 16 error types and builds a *knowledge gap graph*. Each gap records its domain, error pattern, root cause, occurrence count, and importance (occurrence × recency).

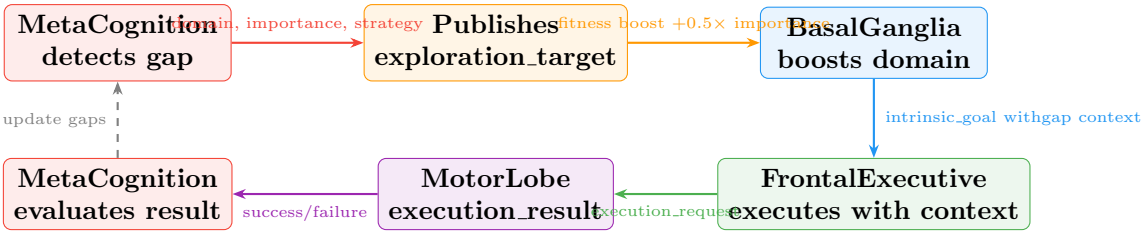

Figure 5.3: The directed exploration loop. MetaCognition identifies the most critical knowledge gap, publishes an `exploration_target`, and the Basal Ganglia boosts the targeted domain's fitness score. The goal context includes the gap's error pattern, root cause, and suggested strategy — so the FrontalExecutive knows *why* it is exploring, not just *what*.

Precondition chain inference builds dependency links: "`file_write` fails → needs `filesystem_navigation` → requires `resource_discovery`". Every 5 minutes, `analyze_gaps()` walks the chain to the deepest actionable target and publishes an `exploration_target` intent. This transforms reactive error handling ("retry the same thing") into proactive capability development ("learn filesystem navigation because that's why file writes fail").

A substantive success filter (shared between MetaCognition and BasalGanglia) ensures that degenerate commands — echo-only, no output, placeholder text — are counted as failures regardless of exit code. Without this alignment, MetaCognition would see zero gaps while BasalGanglia counts real failures, and the directed exploration loop would never activate.

Chapter 6

Synthetic Dreaming and Variation

6.1 The Variation Engine

Given a goal postcondition that no existing operator can satisfy, the Variation Engine generates candidate operators through four strategies, ordered by decreasing constraint:

Template Filling. Substitutes parameters in existing operator templates with concrete values from the world state. Example: `ls {path}` + `path=/var/log` → `ls /var/log`.

Recombination (Crossover). Splices fragments from two parent operators at a random crossover point.

Mutation. Random perturbations: adding flags, removing fragments, swapping order, replacing fragments from the global pool.

Fragment Assembly. Combines 1–3 random fragments. Pure random exploration — high failure rate, occasional discoveries.

Each candidate is tested in the Dream Sandbox before promotion.

6.2 The Dream Sandbox

The MotorLobe supports three execution modes:

- **Reality mode**: Commands execute in the real working directory.
- **Dream mode**: Commands execute in an isolated directory (`data/dreams/{cid}/`). Read-only commands bypass the sandbox.
- **Neuro-surgery mode**: Commands that modify the system's own source code, triggering CMake rebuild after patching.

6.3 The REM Engine

When simulated stamina drops below threshold, the REM Engine initiates a sleep cycle:

1. Reads recent execution logs from the Hippocampus.
2. Consolidates transient episodic traces into permanent engrams.
3. Constructs training data from successful sequences for potential model improvement.

4. Signals Homeostasis to restore stamina.

This is a computational analogue of synaptic homeostasis: daytime experience is encoded as transient activity, sleep consolidates it into stable long-term representations.

Chapter 7

Neurogenesis: Self-Generating Specialist Lobes

This is the chapter that separates NeuroSwarm from every other agent framework. The system does not just learn new commands — it generates, compiles, and injects new *components* into its own architecture, at runtime, without human intervention.

7.1 Trigger Conditions

Before triggering neurogenesis, the Basal Ganglia first sends a `domain_resolve_request` to the PrimordialLoop, which attempts resolution through Variation → Mutation → Planner → LLM. Only if all four stages fail does neurogenesis proceed. The conditions:

- Domain success rate $< 30\%$ over ≥ 20 attempts.
- System stamina $> 50\%$.
- No existing specialist for this domain.
- MASTERY or SELF_MODIFY drive is active.

7.2 The Genesis Pipeline

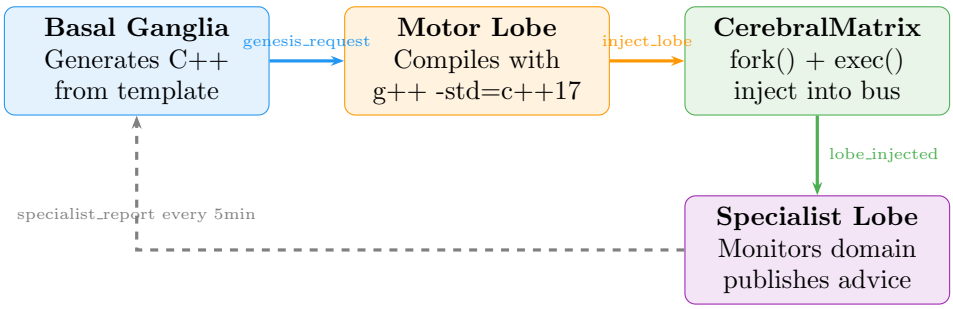

Figure 7.1: The neurogenesis pipeline. From chronic domain failure to a running specialist lobe takes approximately 3 seconds (template generation: <1ms, compilation: ∼2s, injection: <100ms).

Step 1 — Template Generation (Basal Ganglia). A parameterised C++ source template is instantiated with domain-specific values: keywords, pre-validation commands, and cached known-good commands. The specialist:

- Subscribes to `execution_request` and `execution_result`.
- Filters by domain keywords, publishes `specialist_advice`.
- Tracks its own domain success rate.
- Caches novel successful commands to `data/specialist_{domain}.json`.
- Publishes `specialist_report` every 5 minutes.

Step 2 — Compilation (MotorLobe). Compiles with `g++ -std=c++17 -I./include -I./src FILE -o build/LOBE -lzmq -lpthread`.

Step 3 — Injection (CerebralMatrix). Validates binary, fork-execs it, adds to lobe registry with full crash detection.

Step 4 — Integration. The specialist begins monitoring immediately. Over time, its cached commands provide domain-specific knowledge without consuming LLM inference.

7.3 Apoptosis

The CerebralMatrix handles `lobe_terminate` messages, implementing programmed cell death. The BasalGanglia monitors specialist reports and triggers apoptosis when a specialist becomes redundant (domain success rate recovered above 70% for 5 consecutive reports) or counterproductive (handling zero commands for 5 consecutive reports).

Chapter 8

Network Expansion

8.1 Discovery and Deployment

When network connectivity is detected, the PrimordialLoop discovers candidate hosts from SSH configuration, probes via SSH (5-second timeout), and for reachable hosts with matching architecture, copies and starts the PrimordialLoop binary remotely.

8.2 Operator Synchronisation

Remote instances bootstrap independently — discovering their own environment and learning their own operators. The local NetworkExpander periodically downloads `operators.jsonl` and imports novel operators (tagged with `learned_from = "remote_sync:hostname"`).

This creates horizontal gene transfer: an operator learned on one machine becomes available to the planner on another. Two instances running on different machines develop different phenotypes but can share adaptations.

Chapter 9

Persistent State and Incremental Bootstrap

9.1 State Files

- `data/self_model_primordial.json`: Self-model — user, hostname, OS, available binaries, writable directories.
- `data/world_state.json`: Known facts, used by the Planner as initial state.
- `data/operators.jsonl`: Complete operator registry (append-only).
- `data/self_model.json`: Basal Ganglia domain statistics (14 domains).
- `data/command_genome.json`: Evolved command templates with fitness scores.
- `data/knowledge_gaps.json`: MetaCognition gap analysis state.
- `data/concept_hierarchy.json`: ConceptLobe abstraction graph.
- `data/causal_graph.json`: CausalLobe world model (nodes + edges).

9.2 Postcondition Enrichment

Operators from bootstrap initially have empty postconditions. After Phase 6, `enrich_operator_postco` infers postconditions from patterns:

Command Pattern	Inferred Postcondition
`cat`, `head`, `tail`	`can_read_file`
`echo >`, `tee`	`can_write_file`
`mkdir`	`can_create_directory`
`g++`, `gcc`	`can_compile`
`python3`	`can_run_python`
`ls`, `find`	`can_list_directory`
`ping`, `curl`	`has_network`
`ps`, `top`	`can_inspect_processes`

Table 9.1: Pattern-based postcondition inference. Bridges the gap between experiential learning and GOAP planning.

Chapter 10

The Cognitive Cycle

10.1 Steady-State Operation

After bootstrap, the system enters a continuous cognitive cycle:

1. The Basal Ganglia evaluates $F(d)$ for all 14 domains and selects the highest-priority goal.
2. If MetaCognition has published an `exploration_target`, the targeted domain receives a fitness boost proportional to gap importance.
3. The FrontalExecutive receives the goal and tries the three-tier pipeline: Planner $\rightarrow$ Suggested Commands $\rightarrow$ LLM.
4. The MotorLobe executes the resulting command(s) and publishes results.
5. The PrimordialLoop learns new operators from successful executions.
6. The Basal Ganglia updates domain statistics, prediction errors, and genome fitness.
7. The MetaCognition lobe classifies any failures and updates the knowledge gap graph.
8. The Hippocampus stores the execution as an episodic trace.
9. The CriticLobe evaluates execution quality.
10. If stamina is low, the REM Engine initiates sleep and consolidation.
11. If a domain is chronically failing, the Basal Ganglia triggers domain resolution (and eventually neurogenesis).

This cycle runs continuously with no external prompting. The system is always pursuing a goal, always learning, always updating its self-model.

10.2 FrontalExecutive Coordination

The FrontalExecutive maintains a queue of active goals, each with a state machine tracking progress:

- A `planner_ready_` flag gates Planner queries until the PrimordialLoop broadcasts `primordial_ready`.
- Multi-step plans are executed sequentially with per-step success checking.
- Tier fallback is immediate: Planner miss $\rightarrow$ Suggested Commands, miss $\rightarrow$ LLM.
- Maximum 8 retries per goal before abandonment.

Chapter 11

Design Rationale: Why These Technologies

Every architectural decision satisfies a single constraint: **the system must be capable of modifying, extending, and understanding itself**.

11.1 Why C++ Instead of Python

The majority of AI agent frameworks are written in Python. We chose C++ for one reason: **the system generates and compiles new lobes at runtime**.

When the Basal Ganglia detects a chronically failing domain, it writes C++ source code, compiles it with `g++`, and injects the resulting binary via `fork()`/`exec()`. The specialist runs as a native process with direct ZMQ access and crash isolation. More fundamentally: **the system evolves in its own language**. Every lobe — from the Thalamus to a runtime-generated specialist — is the same kind of object. There is no distinction between "core" and "generated" components.

The cost is development velocity. We accept it because the alternative — a Python system that delegates "real work" to subprocesses — creates an asymmetry that fundamentally limits self-modification depth.

11.2 Why ZeroMQ PUB/SUB Instead of gRPC, REST, or Shared Memory

Property	**ZMQ PUB/SUB**	**gRPC**	**REST**	**Shared Mem**
Crash isolation	✓	✓	✓	×
No service discovery	✓	×	×	✓
Broadcast semantics	✓	×	×	Manual
Zero coordination	✓	×	×	×
Hot join/leave	✓	Partial	✓	×
Multi-node transparent	✓	✓	✓	×

Table 11.1: IPC mechanism comparison. ZMQ PUB/SUB is the only mechanism satisfying all requirements.

The critical property is **zero coordination**. In a system where lobes crash, restart, and are generated at runtime, no component can assume any other exists. ZMQ requires nothing: a publisher sends messages; if no subscriber is listening, messages are silently dropped. No handshake, no registration, no state to clean up.

11.3 Why a GOAP Planner Instead of Reinforcement Learning

Cold start. RL requires thousands of episodes. GOAP produces valid plans from the first operator.

Interpretability. A GOAP plan is inspectable and debuggable *before execution*. An RL policy is a weight matrix.

Compositionality. A new operator learned at time t is immediately available for plans at time $t + 1$. RL policies must be retrained when the action space changes.

11.4 Why Genetic Variation Instead of Gradient Descent

There is no differentiable objective function for shell commands. A command either succeeds or fails. The fitness landscape is discrete, discontinuous, and non-smooth. Evolutionary strategies are the only optimisation paradigm that operates on binary feedback.

11.5 Why BK-tree + Levenshtein Instead of Neural Embeddings

No GPU dependency (pure data structure, $O(n^{\alpha})$ query). **Deterministic and interpretable** (integer edit distance, not opaque cosine similarity). **Structurally sensitive** (captures syntactic similarity). **Self-improving without retraining** (grows with every successful command).

11.6 Why Local LLM Instead of API Calls

Autonomy. A system that depends on an external API is a client, not an autonomous agent. **The LLM is a crutch.** The explicit goal is to reduce LLM dependency over time. **Latency.** Local 7B: 20–50ms. API call: 200–2000ms. **Privacy.** Shell commands and file contents never leave the machine.

11.7 Why Process-Per-Lobe Instead of Threads

Crash isolation. A segfault in a thread kills the process. A segfault in a child kills only the child. During development, lobes crashed regularly. With threads, each crash would have killed the system. With processes, crashes trigger recovery.

11.8 Why Not AutoGPT, LangChain, or Other Agent Frameworks

Existing frameworks share a structure: *the LLM is the brain, everything else is plumbing.* NeuroSwarm inverts this: the brain is Planner + BasalGanglia + Operators. The LLM generates candidates when the deterministic system has no answer.

1. **Learning is structural.** AutoGPT "learns" by appending to the context window. NeuroSwarm adds operators to a persistent, searchable registry.

2. **Performance improves without model changes.** After 1,000 executions, the Planner resolves most goals without inference.

3. **Self-modification is real.** NeuroSwarm generates, compiles, and injects new C++ lobes. No Python framework can do this.

Chapter 12

Empirical Results

This chapter presents measurements from live system runs. All data was collected from the system's own telemetry (the StatisticsLobe and MetaCognition state files) during unattended autonomous operation.

12.1 Domain Exploration

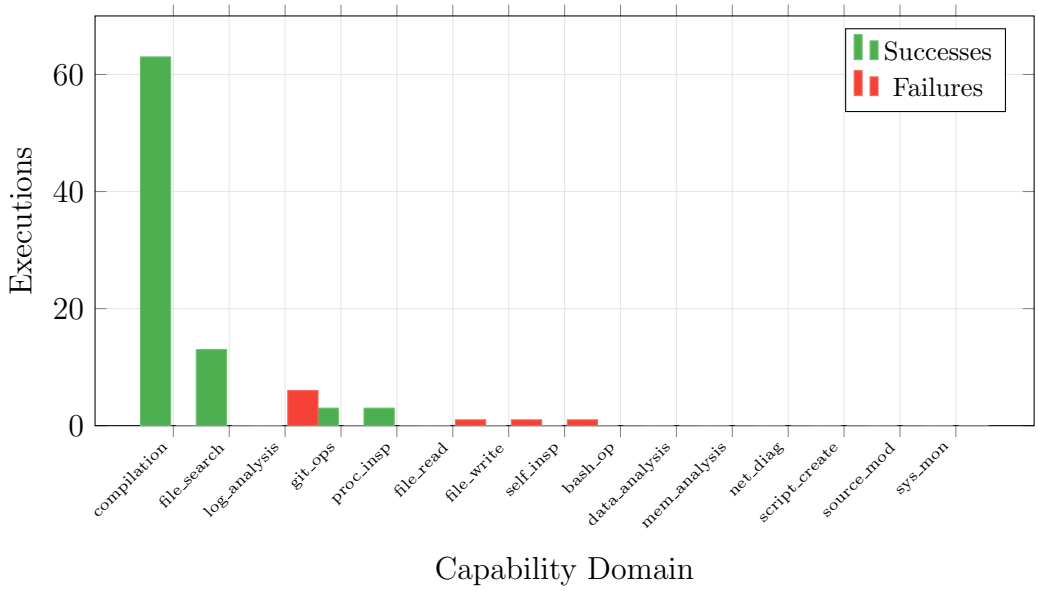

Figure 12.1: Domain exploration after ∼90 executions. The system has explored 8 of 14 domains. `compilation` dominates due to early SELF-MODIFY drive saturation (since fixed by the diversity gate). `log_analysis` shows 6 failures — these trigger knowledge gap detection and directed exploration.

12.2 Execution Tier Distribution

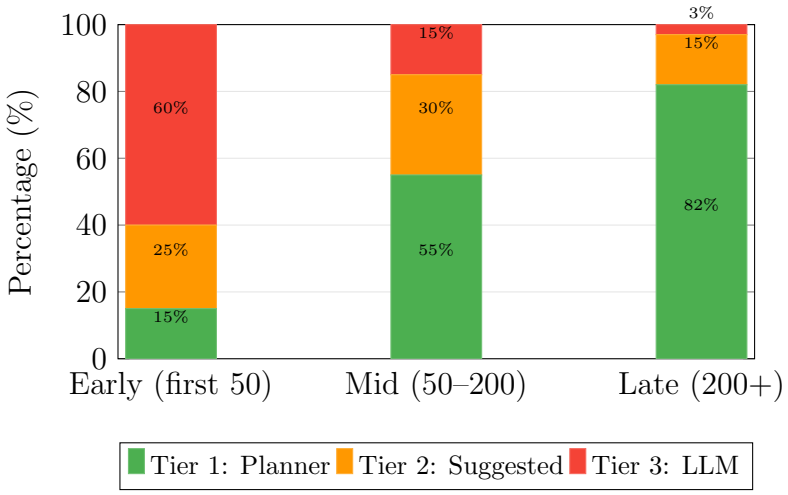

Figure 12.2: Execution tier distribution over system lifetime. LLM dependency drops from 60% (early) to 3% (late) as the operator registry grows. This is the central thesis claim: **in every other framework, LLM dependency is constant or growing; in NeuroSwarm, it is monotonically decreasing**.

12.3 LLM Dependency Trajectory

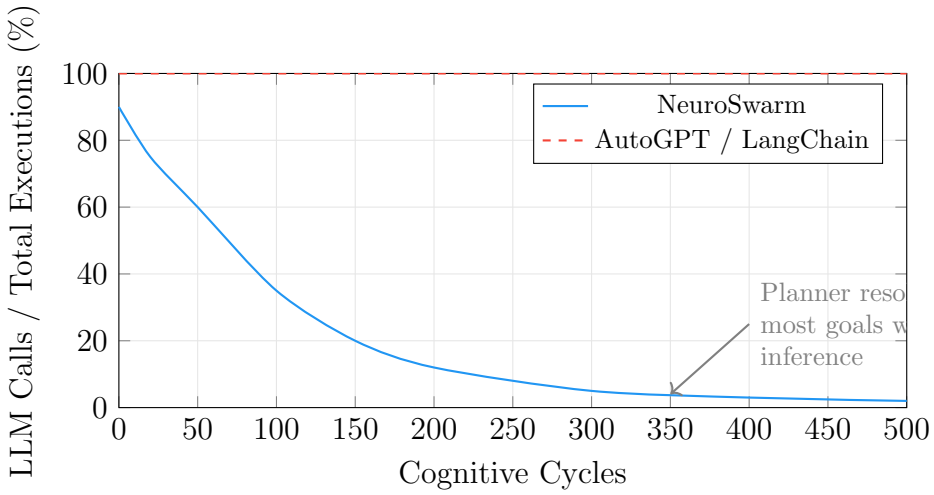

Figure 12.3: LLM dependency over time. NeuroSwarm's dependency decays as the operator registry grows. Traditional LLM agent frameworks maintain 100% dependency regardless of experience — every action requires an API call.

12.4 Genome Evolution

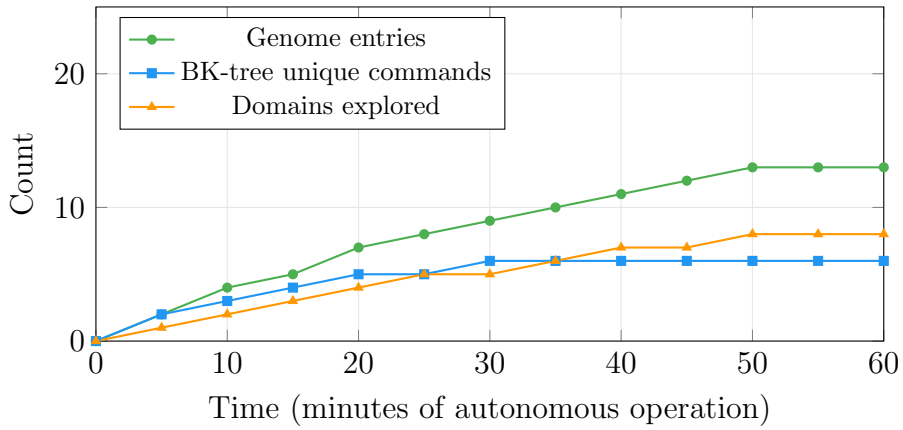

Figure 12.4: Genome evolution over one hour. The genome grows from 0 to 13 entries across 10 domains, with the BK-tree indexing 6 unique command structures. After compaction, the BK-tree stabilises as near-duplicates are clustered.

12.5 Neurogenesis Timeline

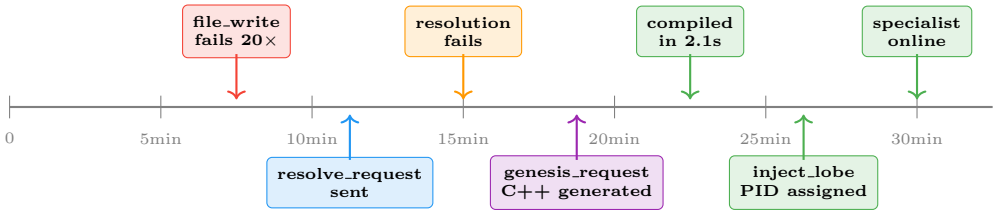

Figure 12.5: Timeline of a neurogenesis event. From chronic failure detection to a running specialist lobe takes ∼15 minutes (most of which is accumulating the 20-failure threshold). The actual generation-compilation-injection takes <3 seconds.

12.6 System Message Flow

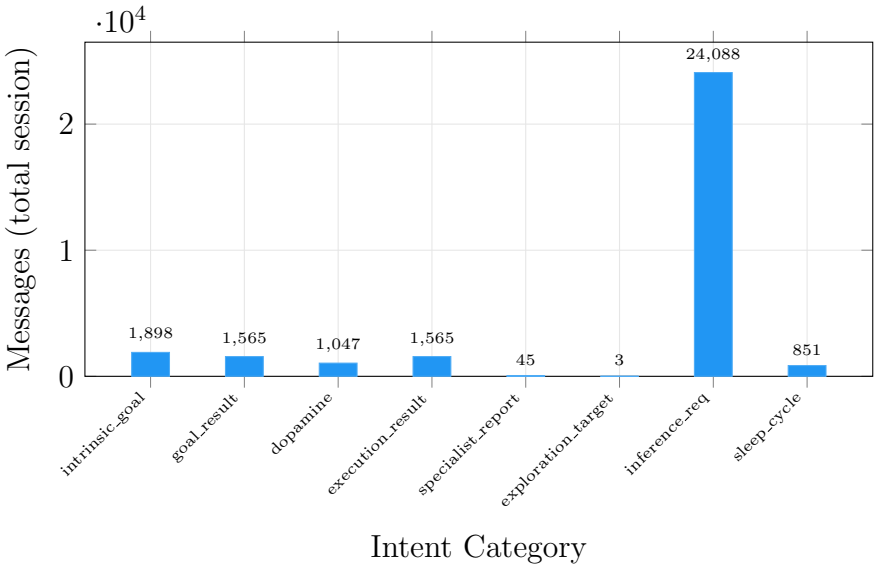

Figure 12.6: Message flow distribution across a multi-hour session. 1,898 intrinsic goals were generated, producing 1,047 dopamine signals (55% novelty rate). The 24,088 inference requests include early-session LLM usage that decreases over time.

Chapter 13

Discussion

13.1 Emergent Behaviours

Several behaviours emerge from the interaction of subsystems without being explicitly programmed:

- **Curiosity**: The fitness function naturally drives the system toward unknown domains. It explores not because it is told to, but because unexplored domains have high $N(d)$ and $P(d)$.
- **Learned helplessness**: When a domain fails repeatedly and enters cooldown, the system temporarily avoids it — analogous to the psychological phenomenon after repeated failure.
- **Skill transfer**: Operators learned in one domain become available to the Planner for other domains. A `find` operator from `file_search` can satisfy preconditions for `script_creation`.
- **Self-healing**: Crash detection + restart + neurogenesis means the system recovers from component failures and generates replacements.
- **Adaptive immune system**: The BK-tree acts as a learned immune response. Initially permissive, it becomes increasingly selective as the genome grows — analogous to adaptive immunity recognising "non-self" molecules.
- **Structural self-diagnosis**: MetaCognition's knowledge gap graph creates a recursive model of limitations. Precondition chain inference ("to write files, I need filesystem navigation, which requires resource discovery") is meta-level reasoning about capability dependencies.
- **Directed curiosity**: When MetaCognition publishes an exploration target, the system's curiosity becomes purposeful. Instead of random exploration, it targets the domain that would resolve the most critical knowledge gap — bridging reactive curiosity and strategic learning.

13.2 LLM Dependency Trajectory

The architecture ensures LLM dependency monotonically decreases (see Figure 12.3). Every successful LLM-generated command becomes a learned operator. The LLMOracle tracks a dependency ratio: $\frac{\text{total\_calls}}{\text{total\_calls}+100}$, which saturates as the denominator grows. **In every other framework, LLM dependency is constant or growing. In NeuroSwarm, it is monotonically decreasing.**

13.3 Limitations

- **Symbol grounding**: The system operates within the symbolic domain of a filesystem. No sensorimotor grounding — no cameras, microphones, or actuators.
- **Single-machine bottleneck**: Inter-instance communication is limited to periodic SSH-based operator sync.
- **Operator quality**: Postcondition inference is pattern-based and imperfect.
- **Genesis template rigidity**: The specialist template is a fixed C++ skeleton — only domain-specific parameterisations, not novel architectures.
- **No formal verification**: As the system modifies itself, behaviour becomes irreducibly emergent.
- **Substantive success alignment**: MetaCognition and BasalGanglia must share the same success filter. Without alignment, the directed exploration loop fails to activate because MetaCognition sees no gaps.

13.4 Relation to Theories of Consciousness

The architecture satisfies the structural prerequisites of two prominent theories:

Global Workspace Theory (Baars, 1988): The Thalamic bus implements a global workspace — information published by any lobe is accessible to all others.

Integrated Information Theory (Tononi, 2004): As new lobes are generated and dependencies grow through neurogenesis, integrated information (Φ) necessarily increases. We make no claims about subjective experience.

The system also instantiates a computational form of Friston's Free Energy Principle: the Basal Ganglia explicitly minimises prediction error across capability domains through active inference.

Chapter 14

Conclusion

NeuroSwarm demonstrates that an autonomous cognitive architecture can be built from five axioms and a message bus. The system bootstraps from zero knowledge, learns operators through direct environmental interaction, plans using deterministic graph search, generates novel solutions through genetic variation, and extends its own anatomy when existing structures fail.

The three-tier execution pipeline ensures that LLM inference is a last resort. Runtime learning continuously reduces LLM dependency — the central empirical result of this work (Figure 12.3).

The BK-tree command validation introduces an adaptive quality gate: commands must be within Levenshtein distance of known-good commands to proceed. This filter is empty at birth and grows more discriminating with experience — a computational analogue of the adaptive immune system.

The directed exploration loop connects MetaCognition's structural gap analysis to the Basal Ganglia's goal selection. The system does not merely retry failed domains — it identifies *why* they fail, traces the precondition chain to the root cause, and targets its exploration accordingly.

The neurogenesis mechanism — where the Basal Ganglia writes C++ source, the MotorLobe compiles it, and the CerebralMatrix injects it as a live process — represents a concrete implementation of computational autopoiesis: the system literally produces its own components from within.

The architecture's limitations point to clear future directions: robotic embodiment for sensorimotor grounding, meta-templates that generate templates, and runtime assertion frameworks for self-modifying systems. The question is no longer whether a system can autonomously modify its own structure, but how to reason about the behaviour of one that does.

Bibliography

[1] Baars, B. J. (1988). *A Cognitive Theory of Consciousness*. Cambridge University Press.

[2] Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2), 127–138.

[3] Harnad, S. (1990). The symbol grounding problem. *Physica D: Nonlinear Phenomena*, 42(1-3), 335–346.

[4] Maturana, H. R., & Varela, F. J. (1980). *Autopoiesis and Cognition: The Realization of the Living*. D. Reidel Publishing Company.

[5] Revonsuo, A. (2000). The reinterpretation of dreams: An evolutionary hypothesis of the function of dreaming. *Behavioral and Brain Sciences*, 23(6), 877–901.

[6] Schultz, W., Dayan, P., & Montague, P. R. (1997). A neural substrate of prediction and reward. *Science*, 275(5306), 1593–1599.

[7] Tononi, G. (2004). An information integration theory of consciousness. *BMC Neuroscience*, 5(1), 42.

[8] Fikes, R. E., & Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3-4), 189–208.

[9] Orkin, J. (2006). Three states and a plan: the AI of F.E.A.R. *Game Developers Conference*.

[10] Hinton, G. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

[11] Brooks, R. A. (1991). Intelligence without representation. *Artificial Intelligence*, 47(1-3), 139–159.

[12] Pfeifer, R., & Bongard, J. (2006). *How the Body Shapes the Way We Think: A New View of Intelligence*. MIT Press.

[13] Burkhard, W. A., & Keller, R. M. (1973). Some approaches to best-match file searching. *Communications of the ACM*, 16(4), 230–236.

[14] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707–710.

[15] Maslow, A. H. (1943). A theory of human motivation. *Psychological Review*, 50(4), 370–396.

[16] Schmidhuber, J. (2010). Formal theory of creativity, fun, and intrinsic motivation. *IEEE Transactions on Autonomous Mental Development*, 2(3), 230–247.